

## ATIVIDADE: JUNIT E INJEÇÃO DE DEPENDÊNCIAS

### Questão 1

Crie uma classe de teste que valide os métodos da classe Usuário considerando:

1. Campos do tipo String não podem ser nulos ou vazios;
2. Nome não pode conter caracteres especiais ou números, sendo permitido apenas espaço entre as palavras;
3. E-mail deve estar no seguinte padrão usuario@dominio;
4. usuario pode conter apenas caracteres alfanuméricos não acentuados, ponto (.) ou sublinhado;
5. dominio pode conter apenas caracteres alfanuméricos;

```
@Test
void testCamposNulosOuVazios() {
    assertThrows(IllegalArgumentException.class, () -> usuario.setNome(null));
    assertThrows(IllegalArgumentException.class, () -> usuario.setNome(""));
    assertThrows(IllegalArgumentException.class, () -> usuario.setNome("  "));

    assertThrows(IllegalArgumentException.class, () -> usuario.setEmail(null));
    assertThrows(IllegalArgumentException.class, () -> usuario.setEmail(""));

    assertThrows(IllegalArgumentException.class, () -> usuario.setSenha(null));
    assertThrows(IllegalArgumentException.class, () -> usuario.setSenha(""));
}

@ParameterizedTest
@ValueSource(strings = {"Joao Silva", "Maria", "Ana Paula Souza"})
void testNomeValido(String nomeValido) {
    assertDoesNotThrow(() -> usuario.setNome(nomeValido));
    assertEquals(nomeValido, usuario.getNome());
}

@ParameterizedTest
@ValueSource(strings = {"Joao123", "Maria_Silva", "Jose!", " "})
void testNomeInvalido(String nomeInvalido) {
    assertThrows(IllegalArgumentException.class, () -> usuario.setNome(nomeInvalido));
}
```

```

@ParameterizedTest
@ValueSource(strings = {"user.name_1@dominio", "teste@empresa", "joao.silva@provedor"})
void testEmailValido(String emailValido) {
    assertDoesNotThrow(() -> usuario.setEmail(emailValido));
    assertEquals(emailValido, usuario.getEmail());
}

@ParameterizedTest
@ValueSource(strings = {
    "usuario@dominio@extra",
    "usuario.com",
    "user!@dominio",
    "user@domín-io",
    "user@dom.in.io",
    "user á@dominio"
})
void testEmailInvalido(String emailInvalido) {
    assertThrows(IllegalArgumentException.class, () -> usuario.setEmail(emailInvalido));
}
}

```

## Questão 2

Complete o código de Usuário para fornecer os métodos pendentes (ex. get/set) e as funcionalidades de validação a serem usadas na classe de teste (ex. validarNome).

```

public void validarNome(String nome) { 1 usage
    validarString(nome);
    if (!Pattern.matches(REGEX_NOME, nome)) {
        throw new IllegalArgumentException();
    }
}

public void validarEmail(String email) { 1 usage
    validarString(email);
    if (!email.contains("@")) {
        throw new IllegalArgumentException();
    }

    String[] partes = email.split(regex: "@", limit: -1);
    if (partes.length != 2) {
        throw new IllegalArgumentException();
    }

    if (!Pattern.matches(REGEX_USER_EMAIL, partes[0]) || !Pattern.matches(REGEX_DOM_EMAIL, partes[1]))
        throw new IllegalArgumentException();
}
}

```

## Questão 3

A autenticação sem sucesso deve ser permitida por até três vezes dentro de 30 segundos, devendo lançar exceção do tipo "ExceededAttemptsException" se 4 ou mais tentativas forem realizadas em menos de 30 segundos. Neste caso, novas tentativas deverão ser permitidas apenas depois de uma espera de um minuto.

```
public void autenticar(String senhaEntrada) throws ExceededAttemptsException {
    long agora = System.currentTimeMillis();

    if (agora < bloqueadoAte) {
        throw new ExceededAttemptsException();
    }

    validarString(senhaEntrada);

    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hashBytes = digest.digest(senhaEntrada.getBytes());
        String hashEntrada = Base64.getEncoder().encodeToString(hashBytes);

        if (hashEntrada.equals(this.senhaHash)) {
            falhasTimestamp.clear();
            bloqueadoAte = 0;
        } else {
            registrarFalha(agora);
        }
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException(e);
    }
}
```

```
class ExceededAttemptsException extends Exception { 4 usages
    public ExceededAttemptsException() { 2 usages
        super("Limite de tentativas excedido. Tente novamente mais tarde.");
    }
}
```

#### Questão 4

Adapte o código para que essa funcionalidade (descrita na questão 3) seja devidamente implementada.

```

public void autenticar(String senhaEntrada) throws ExceededAttemptsException {
    long agora = System.currentTimeMillis();

    if (agora < bloqueadoAte) {
        throw new ExceededAttemptsException();
    }

    validarString(senhaEntrada);

    String hashEntrada = gerarHash(senhaEntrada);

    if (hashEntrada.equals(this.senhaHash)) {
        falhasTimestamp.clear();
        bloqueadoAte = 0;
    } else {
        registrarFalha(agora);
    }
}

```

```

private void registrarFalha(long agora) throws ExceededAttemptsException {
    falhasTimestamp.add(agora);
    falhasTimestamp.removeIf( Long t -> agora - t > 30000);

    if (falhasTimestamp.size() >= 4) {
        bloqueadoAte = agora + 60000;
        throw new ExceededAttemptsException();
    }
}

```

## Questão 5

Utilize as classes TestInfo, TestReporter e RepetitionInfo para tornar mais claro os resultados dos testes, em especial se houver falhas.

```

public class UsuarioAuthTest {
    private Usuario usuario; 7 usages

    @BeforeEach
    void setUp() {
        usuario = new Usuario();
        usuario.setNome("Joao Silva");
        usuario.setEmail("joao@provedor");
        usuario.setSenha("senha123");
    }
}

```

```

@RepeatedTest(value = 4, name = "{displayName} - Tentativa {currentRepetition} de {totalRepetitions}")
@DisplayName("Validacao de bloqueio por tentativas excessivas")
void testBloqueioPorTentativas(TestInfo testInfo, TestReporter testReporter, RepetitionInfo repetitionInfo) {
    testReporter.publishEntry("Executando", testInfo.getDisplayName());
    testReporter.publishEntry("Repeticao", String.valueOf(repetitionInfo.getCurrentRepetition()));

    if (repetitionInfo.getCurrentRepetition() < 4) {
        assertThrows(ExceededAttemptsException.class, () -> {
            try {
                usuario.autenticar(senhaEntrada: "senha_errada");
            } catch (ExceededAttemptsException e) {
                throw e;
            } catch (Exception e) {
                testReporter.publishEntry("Erro", "Capturada excecao inesperada");
            }
        });

        int falhasEsperadas = repetitionInfo.getCurrentRepetition();
        testReporter.publishEntry("Status", "Falha " + falhasEsperadas + " registrada com sucesso");
    } else {
        ExceededAttemptsException exception = assertThrows(ExceededAttemptsException.class, () -> {
            usuario.autenticar(senhaEntrada: "senha_errada");
        });
    }
}

```

```

    } else {
        ExceededAttemptsException exception = assertThrows(ExceededAttemptsException.class, () -> {
            usuario.autenticar(senhaEntrada: "senha_errada");
        });

        assertEquals(expected: "Limite de tentativas excedido.", exception.getMessage());
        testReporter.publishEntry("Resultado", "Utilizador bloqueado corretamente na quarta tentativa");
    }
}

```

```

@Test
@DisplayName("Validacao de sucesso na autenticacao")
void testAutenticacaoSucesso(TestInfo testInfo, TestReporter testReporter) {
    testReporter.publishEntry("Teste", testInfo.getDisplayName());

    assertDoesNotThrow(() -> {
        usuario.autenticar(senhaEntrada: "senha123");
    });

    testReporter.publishEntry("Info", "Autenticacao realizada com sucesso");
}
}

```